오픈소스 기반 데이터분석

07강

데이터 전처리 2

컴퓨터과학과 정재화교수



학습목차

1. pandas를 이용한 데이터 정제
2. pandas를 이용한 데이터 문제해결



01

pandas를 이용한 데이터 정제



데이터 정제

- 데이터 정제(data cleansing)의 정의
  - 결측치, 이상치, 중복값, 오류값 등 데이터 분석에 방해가 되는 불완전하거나 부정확한 데이터를 탐지하고 수정·제거하는 과정
  - 대표적으로 데이터 전처리 과정에서 수행
- 주로 결측치, 이상치를 식별하고 적절하게 처리하는 과정으로 구성
  - 결측치: 결측치는 데이터에 값이 존재하지 않는 상태
  - 이상치: 일반적인 범위를 벗어난 특이한 값

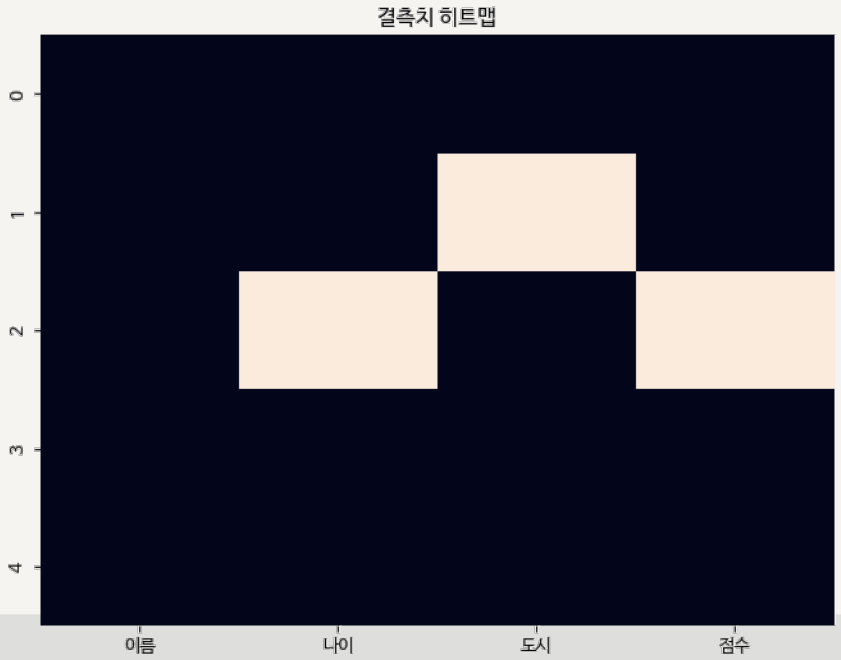
결측치 찾기

- Pandas에서는 결측치는 NaN(Not a Number)로 표현
  - NumPy에 포함
- Pandas의 isnull(), isna() 메소드
  - 각 요소가 결측치인지 여부를 나타내는 불리언 값을 반환
  - sum() 메소드와 결합, DataFrame 내 결측치 개수 파악
  - any() 메소드와 결합
    - DataFrame내 결측치 포함 여부 파악
    - 결측치의 완전한 처리 여부 파악
- notnull( ) 메서드
  - isnull( )의 반대 결과를 제공

시각화를 통한 결측치 찾기

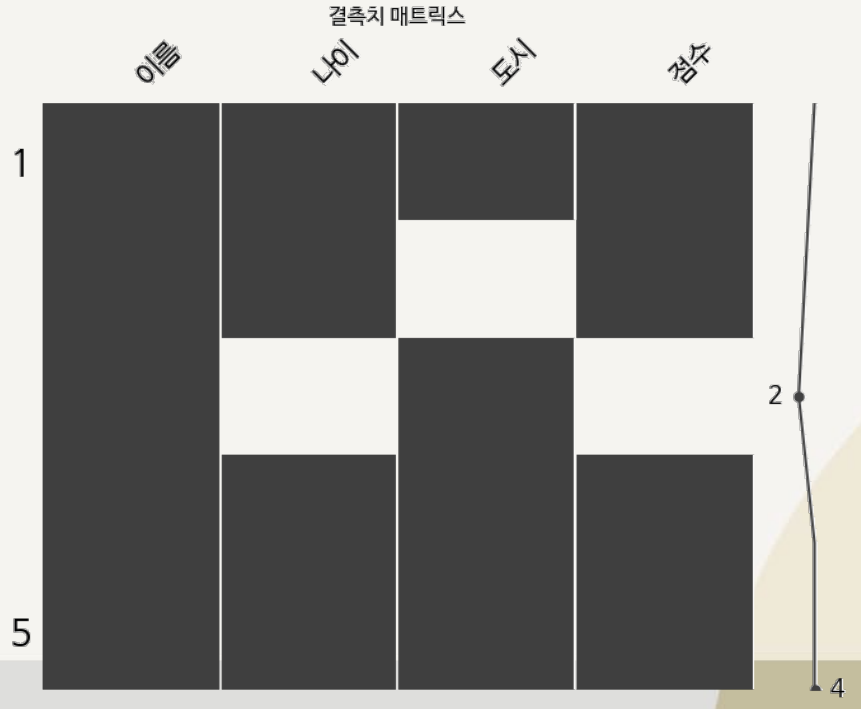
히트맵

- seaborn 라이브러리의 heatmap() 함수 사용
- 변수 간의 결측치 관계를 한눈에 확인



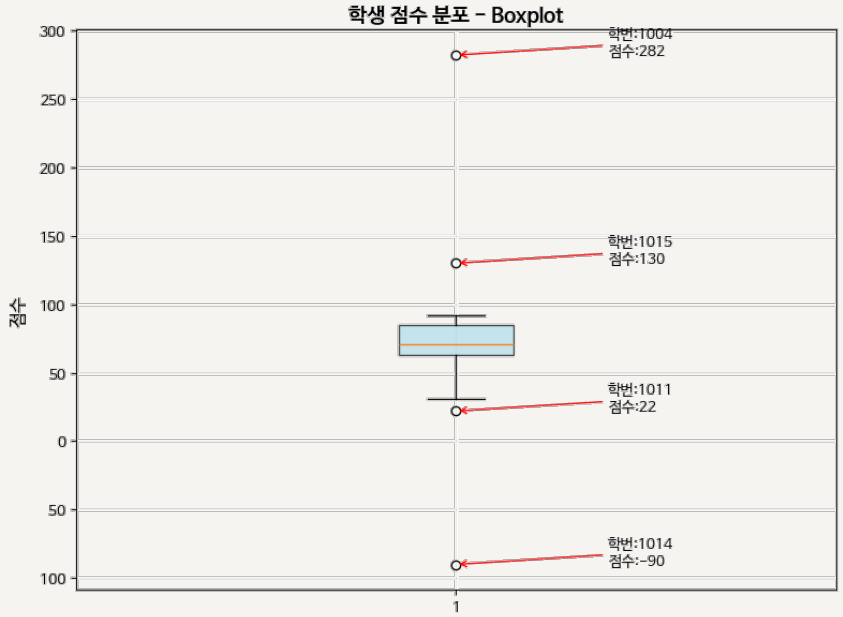
매트릭스

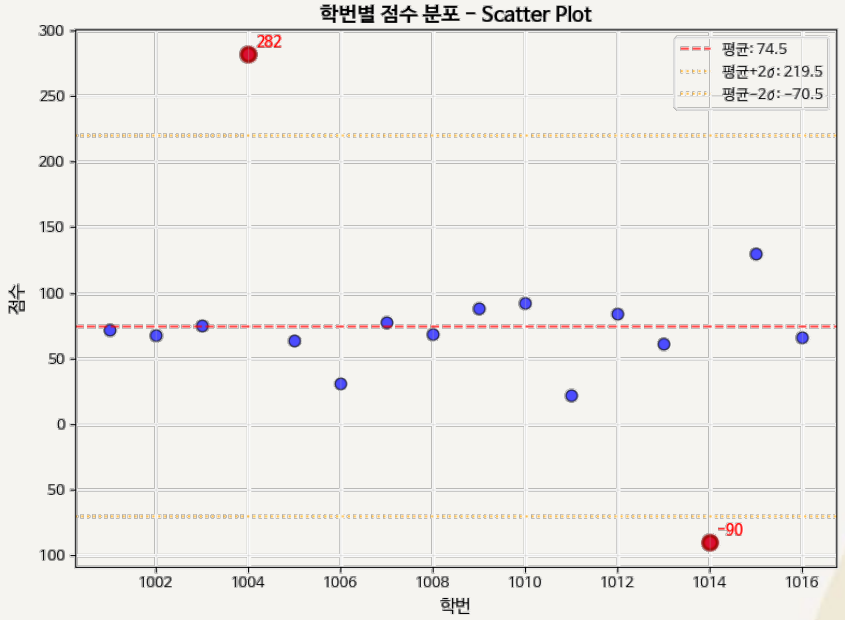
- missingno 라이브러리의 matrix() 함수 사용
- 데이터의 누락 여부를 행렬 형태로 표현



이상치 찾기

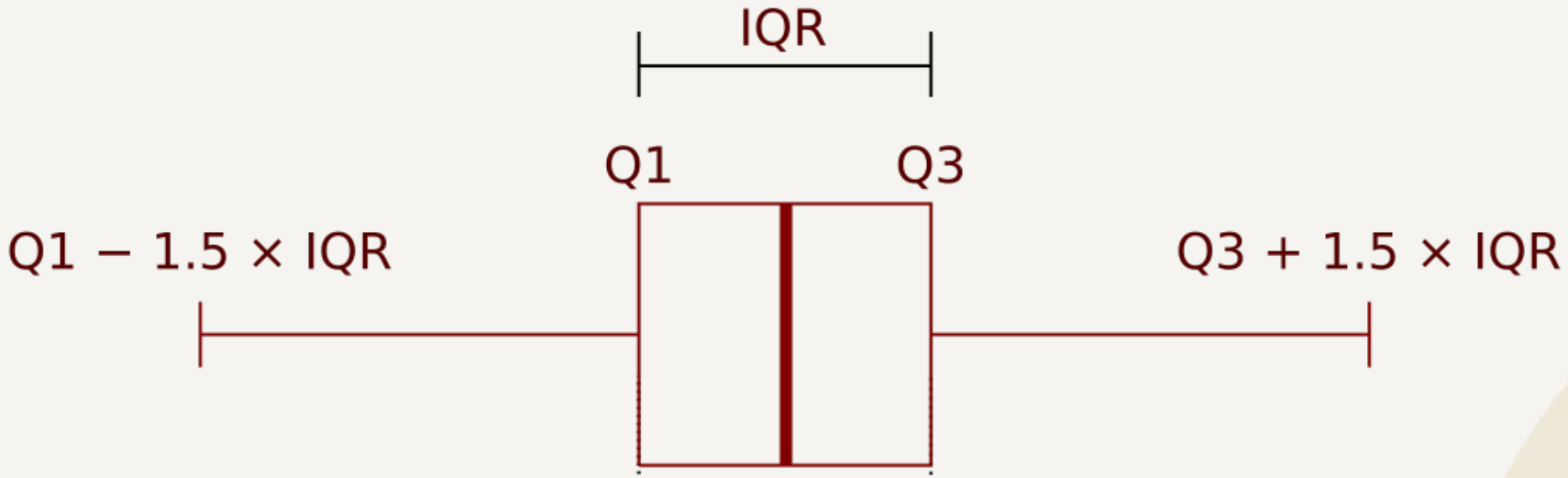
- 통계적 방법
  - IQR(사분위범위), Z-Score 등의 통계값 사용
- 시각적 방법
  - 박스플롯, 산점도 등의 그래프 사용





IQR(Interquartile Range)를 이용한 이상치 찾기

- 사분위수를 기준으로 하여 데이터 분포의 중심 50% 범위를 계산하고, 이 범위를 벗어나는 데이터를 이상치로 간주하는 방식
- 비정규 또는 이상치가 많은 데이터에 적합



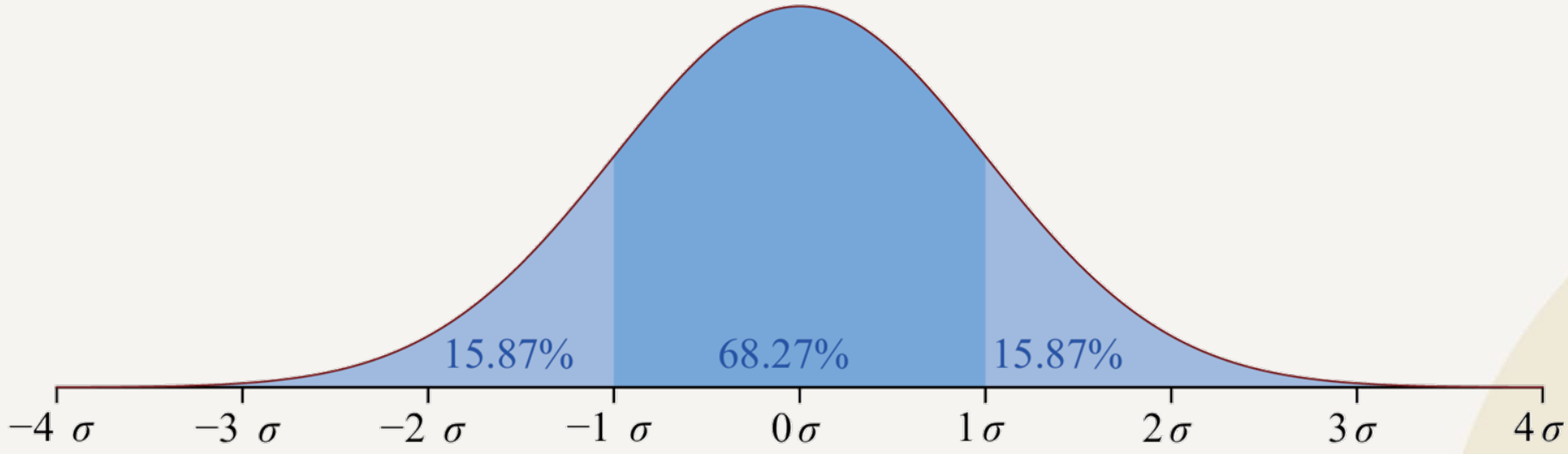
IQR을 통한 이상치 탐색 예시

```
점수_데이터 = [72, 68, 75, 282, 64, 31, 78, 69, 88, 92, 22, 84,
 61, -90, 130, 66]
학번_데이터 = list(range(1001, 1001 + len(점수_데이터)))
df = pd.DataFrame({ '학번': 학번_데이터, '점수': 점수_데이터
})
q1 = df['점수'].quantile(0.25)
q3 = df['점수'].quantile(0.75)
iqr = q3 - q1 # IQR 계산
하한값 = q1 - 1.5 * iqr # 하한 경계값
상한값 = q3 + 1.5 * iqr # 상한 경계값
print("IQR 통계량:")
print(f"Q1 (25% 지점): {q1:.2f}")
print(f"Q3 (75% 지점): {q3:.2f}")
print(f"IQR (Q3 - Q1): {iqr:.2f}")
print(f"하한 경계값 (Q1 - 1.5*IQR): {하한값:.2f}")
print(f"상한 경계값 (Q3 + 1.5*IQR): {상한값:.2f}")
```

Z-Score(Z-점수)를 이용한 이상치 찾기

- 정규분포를 가정하고 각 데이터가 평균으로부터 얼마나 떨어져 있는지를 표준편차 단위로 측정하는 방식
- 정규분포에서 강력하나 평균/표준편차에 민감

Z-Score = |(데이터 값 - 평균) / 표준편차|



Z-score를 통한 이상치 탐색 예시

```
점수_평균 = df['점수'].mean()
점수_표준편차 = df['점수'].std()
df['점수_Z'] = (df['점수'] - 점수_평균) / 점수_표준편차

threshold = 2
df['이상치여부'] = df['점수_Z'].abs() > threshold

print("학생 성적 Z-score 분석 결과:")
print("=" * 50)
print(df)
print("\n이상치로 판별된 데이터:")
print("=" * 50)
print(df[df['이상치여부'] == True])
이상치_비율 = df['이상치여부'].mean() * 100
print(f"\n이상치 비율: {이상치_비율:.2f}%")
```

02

pandas를 이용한 데이터 문제 해결



데이터 문제 해결 전략

- 데이터의 특성, 문제의 유형, 분석 목적, 그리고 데이터의 크기 등 여러 요소를 고려하여 선택
- 주요 문제 해결 전략
  - 제거: 결측치나 이상치를 포함하는 행/열을 삭제
    - dropna() 메소드를 사용하여 행이나 열을 제거
    - 정보의 손실 가능성으로, 데이터셋의 규모가 작거나 결측치가 특정 패턴을 가지고 있는 경우 사용에 신중
  - 대치: 결측치나 이상치를 적절한 값으로 대체
    - 데이터 손실을 최소화하면서 데이터의 활용도 향상
  - 보간
    - 주변 데이터 포인트의 관계를 이용하여 결측치를 추정

대치 전략

대체 방법	설명	적용 대상
평균값 대치	결측치를 해당 열의 평균값으로 대체	연속형 변수
중앙값 대치	결측치를 해당 열의 중앙값으로 대체	연속형 변수
최빈값 대치	결측치를 해당 열에서 가장 빈번하게 나타나는 값으로 대체	범주형 변수
특정값 대치	결측치를 사용자가 지정한 특정 값으로 대체	모든 유형
전방/후방 채우기	이전/이후 관측값으로 결측치를 대체	시계열 데이터
k-최근접 이웃 대치	유사한 샘플을 기반으로 결측치를 추정	다변량 관계가 있는 데이터

보간 전략

보간 방법	설명	특징 및 장단점
선형 보간	결측치 전후의 두 데이터 포인트를 직선으로 연결하여 추정	기본 보간 방법으로 간단하고 계산이 빠름, 그러나 급격한 변화가 있는 데이터에서는 부정확
다항식 보간	주변 데이터 포인트를 다항식으로 근사하여 추정	더 부드러운 추정이 가능하나, 과적합 위험이 있음
스플라인 보간	여러 개의 다항식을 연결하여 결측치를 추정	복잡한 패턴을 더 잘 포착할 수 있으나 계산 비용이 높음
시간 기반 보간	시계열 데이터에서 시간 간격을 고려하여 보간	시간 간격을 반영할 수 있어 시계열 데이터에 적합하지만, 일정한 간격이 아닌 경우 부정확

pandas 데이터 값 변경 메소드

- replace()
  - 특정 값을 다른 값으로 바꿀 때 사용
- map()
  - 각 요소에 함수나 딕셔너리를 적용하여 값을 변경
- apply()
  - 데이터의 각 요소에 함수를 적용할 때 사용
- loc 인덱서
  - 특정 조건을 만족하는 행이나 열을 선택하고, 값을 변경
- where()
  - 특정 조건을 만족하는 값만 유지하거나, 다른 값으로 변경

pandas 데이터 값 변경의 예

```
 1 import pandas as pd
 2
 3 data = {'age': [25, 30, None, 22, 35],
 4         'city': ['Seoul', None, 'Incheon', 'Seoul', 'Daejeon'],
 5         'score': [90, 85, None, 80, 92]}
 6 df = pd.DataFrame(data)
 7
 8 df['city'] = df['city'].replace('Seoul', '서울')
 9 df['age_str'] = df['age'].map(lambda x: f" {x}살"
                        if pd.notna(x) else "알수없음")
10 print("\nmap함수를 이용한 age 변경:\n",df['age_str'])
11 df.loc[df['score'] < 90, 'score'] = 90
12 df['age_where'] = df['age'].where(df['age']>=30,
                                    other = 0)
```

날짜 데이터 다루기

- 데이터 분석을 시작 전, 모든 날짜 데이터를 일관된 형식으로 변환
  - 국가, 조직, 문화별로 다른 날짜 표현식 및 패턴을 사용
  - 윤년이나 월별 일수 차이와 같은 복잡한 특성

```
1 date_str = ['2025-07-01', '2025-08-01', '2025-09-01']
2 df_date = pd.DataFrame({'date_str':date_str})
3 df_date['date'] = pd.to_datetime(df_date['date_str'])
4 df_date['month'] = df_date['date'].dt.month
5 df_date['weekday'] = df_date['date'].dt.day_name( )
6 df_date['date_ymd'] =
          df_date['date'].dt.strftime('%Y년 %m월 %d일')
7 df_date['date_weekday'] =
          df_date['date'].dt.strftime('%A, %Y-%m-%d')
```

개념 정리 실습

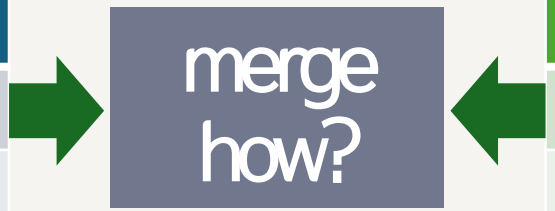
- Kaggle의 "Store Sales – Time Series Forecasting" 데이터셋을 활용하여 데이터 전처리 과정 수행
  - Kaggle은 데이터 과학 플랫폼으로, 데이터 과학자들이 경쟁하고 지식을 공유하는 생태계

단계	주요 내용	사용 도구 / 방법
1. 데이터 로드 및 측정	- kaggle 가입 및 API 키 발급 - 데이터셋 다운로드 - 데이터 구조 파악	- pandas 라이브러리 - describe(), info() 메서드
2. 결측치 처리	- 결측치 탐지 및 보간법 적용	- isnull(), isna() 함수 - interpolate() 메서드
3. 이상치 처리	- 이상치 탐지 및 처리	- IQR 방법
4. 날짜 데이터 처리	- 날짜 형식 변환 - 날짜 특성 추출	- to_datetime() 함수 - dt 접근자
5. 데이터 통합	- 다양한 데이터셋 병합	- merge() 함수

개념 정리 실습

- 데이터 통합: 서로 다른 DataFrame을 결합
  - merge 메소드는 SQL의 JOIN과 유사한 방식으로 두 DataFrame을 특정 키를 기준으로 결합

ID	X
1	a1
2	a2



ID	Y
2	b1
3	b2

ID	X	Y
2	a2	b1

inner join

ID	X	Y
1	a1	NaN
2	a2	b1
3	NaN	b2

outer join

ID	X	Y
1	a1	NaN
2	a2	b1

left join

ID	X	Y
2	a2	b1
3	NaN	b2

right join

정리하기

- 데이터 문제 해결 방법은 다양하며, 상황에 맞는 적절한 방법을 선택해야 한다.
- 데이터 전처리는 많은 시간과 노력이 소모되므로 효율적인 전략이 중요하다.
- describe(), info() 함수를 통해 데이터의 기술 통계량과 구조를 파악한다.
- isnull(), isna() 함수를 사용하여 결측치를 탐지한다.
- interpolate(), fillna() 함수를 사용하여 결측치를 처리한다.
- IQR 방식이나 Z-점수를 활용하여 이상치를 탐지하고 처리한다.
- to_datetime() 함수와 dt 접근자를 사용하여 날짜 데이터를 변환하고 속성을 추출한다.
- merge() 함수를 사용하여 데이터셋을 통합한다.

08강 데이터 분석 1

다음시간안내

